PAYMENT INTEGRATION PROJECT ACTUAL STEPS AND REASONS

When a customer clicks the **Pay** button and starts the Stripe Checkout process, your frontend calls your backend to create a Checkout Session securely. Your backend communicates with Stripe using your **test or live API key** and sends relevant data such as amount, currency, customer information, and the success/return URLs. Stripe responds with a **Checkout Session object**, which is a JSON object containing many fields, including the url that your frontend uses to redirect the customer to the Stripe-hosted payment page. The Checkout Session object also includes id (unique session ID), object (type), amount_total, currency, payment_status, customer (optional), success_url, cancel_url, and payment_intent. While the frontend mainly needs the url for redirection, your backend can store the entire object for tracking, reconciliation, and linking with webhook events. From this point, Stripe takes over the payment flow: it handles card input, authentication, fraud checks, and communication with banks or card networks to authorize and capture the payment. The **success URL** that you provided is purely for user experience; it only tells the customer that they reached the end of the payment flow and should **never** be relied upon as confirmation of payment. Stripe may complete the payment even if the user closes the browser or navigates away.

Once Stripe finalizes the payment with the bank, it notifies your backend by sending a **webhook**. Technically, a webhook is just an **HTTP POST request** containing a JSON **payload**, which includes the event type (for example, checkout.session.completed), event ID, and all relevant session/payment information. The POST request also contains a signature header (Stripe-Signature) that your backend

must verify to ensure authenticity. The webhook is the true source of payment confirmation and is independent of the success URL.

The backend function you write to handle this webhook serves as both an **API endpoint** and an **event handler**. It is the **URL you configure in the Stripe Dashboard** when setting up webhooks, so Stripe knows where to send the POST requests.

Ah, I see the confusion — it's about **what "configuring the endpoint" really means in development vs production**. Let me clarify carefully.

1️⃣ In production (public backend)

- Your backend has a **public URL** like https://myapp.com/stripe/webhook.
- You **configure this URL in Stripe Dashboard** as the webhook endpoint.
- Stripe can **directly send POST requests** (webhooks) to this URL.
- No middleman is needed; your function receives the webhook directly.

2️⃣ In local development (private backend on localhost)

- Your backend is running on http://localhost:4242/stripe/webhook, which Stripe **cannot reach directly**, because localhost is not publicly accessible from the internet.
- You still need a **way for Stripe to know where to send webhooks**, and this is where the **Stripe CLI** comes in:
    1. You run the CLI and tell it to **listen for Stripe events** and forward them to your local endpoint:

2. stripe listen --forward-to localhost:4242/stripe/webhook
3. The CLI gives you a **webhook signing secret** that you use in your local server to verify signatures.
4. Stripe sends events to the CLI, and the CLI **forwards the POST requests to your local function**.

- So even in local development, you are technically "configuring an endpoint" in Stripe Dashboard (or via CLI) — it's just that instead of sending directly to your local machine, Stripe sends it to the CLI, which acts as a bridge.

✅ **Key point:**

- **Configuring the endpoint** in Stripe is always required.
  - In production: it points directly to your public backend URL.
  - In development/local: it points to the Stripe CLI listener, which forwards to your local function.
- Stripe **never sends webhooks directly to localhost** by itself. The CLI or another tunneling tool (like ngrok) is required to bridge the connection.

In short: **you configure the endpoint in Stripe Dashboard** no matter what, but **how the webhook reaches your backend depends on whether your backend is public or local**. The CLI just makes your local backend look like a public endpoint to Stripe.

When Stripe sends the webhook to your endpoint, your function first verifies the signature, then reads the event type, and finally executes the fulfilment logic. "Fulfilling the order" means your backend takes

the payment information from Stripe and performs all business-specific actions, such as marking the order as paid in the database, generating invoices, sending confirmation emails, unlocking digital content, or triggering shipment for physical products. Stripe itself does not perform any of these actions; it only notifies your backend that the payment succeeded.

Your webhook handler should be **idempotent**, which means it can safely handle the same event multiple times without causing duplication or errors. This is important because Stripe may retry sending a webhook if your server does not respond successfully or if there is a network/server failure. In practice, this means checking the event.id or session.id in your database and only processing it if it hasn't been handled before.

After processing the webhook successfully, your backend must respond with a **200 OK** status to Stripe. This tells Stripe that the webhook was received and handled correctly. If your server is down or the network fails, Stripe will automatically retry sending the webhook according to its retry schedule. Your idempotent handler ensures that repeated events do not cause duplicate orders, invoices, or other side effects.

Stripe CLI in the workflow

The **Stripe CLI** is a command-line tool used **during development and testing** to simulate Stripe events, test webhooks locally, and debug your integration. It is **only for testing/development** and is not needed in production.

Where it fits in the flow:

- After creating your backend webhook function (the API endpoint that processes webhooks).

- Before deploying to a public server, to test that your webhook handler works correctly.
- After creating a Checkout Session in test mode, to simulate Stripe sending events like checkout.session.completed.
- When you want to debug webhook handling, inspect payloads, or test retries and idempotency.

Why we sometimes use Stripe CLI instead of sending webhooks directly to our function:

- When developing locally, your backend is running on localhost, which Stripe **cannot reach directly**, so you cannot receive webhooks from Stripe directly.
- The Stripe CLI **listens for Stripe events and forwards them** to your local server, acting as a bridge.
- This allows you to **test and debug webhooks locally** without deploying or exposing your server publicly.

Flow with CLI:

Stripe event → Stripe CLI (on your machine) → your local webhook function

In production:

- Once your backend is deployed on a **public URL**, you can configure that URL directly in the Stripe Dashboard.
- Stripe will then send **POST requests directly** to your function without needing the CLI:

Stripe event → your backend webhook endpoint

Practical steps with Stripe CLI:

1. Install the CLI:
2. brew install stripe # macOS

3. npm install -g stripe-cli # alternative
4. Login to Stripe:
5. stripe login
6. Forward webhook events to your local server:
7. stripe listen --forward-to localhost:4242/stripe/webhook
    - The CLI provides the webhook signing secret for local signature verification.
8. Trigger test webhook events:
9. stripe trigger checkout.session.completed
    - Sends realistic test events to your local backend endpoint.
10. Inspect payloads and logs to ensure your function handles events correctly.
11. After confirming everything works, deploy your webhook endpoint to production and stop using CLI forwarding.

From a practical perspective, after completing a test payment and being redirected to the success URL, you must:

1. Configure the webhook endpoint in Stripe (your backend API URL or CLI listener in local development).
2. Implement your backend function to receive and process the POST request.
3. Verify the webhook signature.
4. Execute fulfilment logic (mark orders paid, send invoices, trigger shipment, etc.).
5. Respond with 200 OK to Stripe.

6. Use Stripe CLI during development to simulate events, test idempotency, and debug payload handling.

In short, the complete flow is: the customer clicks **Pay** → your backend creates a Checkout Session → Stripe responds with the **full session object** (including the URL) → frontend redirects the customer to Stripe → Stripe handles the payment with the bank → Stripe sends a **POST request webhook** to the URL you exposed (your API function) → your backend function reads the payload, verifies it, processes fulfilment, responds with 200 OK, and handles retries if needed. Using the **Stripe CLI** in development ensures your webhook handler works reliably before deploying, while in production, Stripe sends webhooks **directly to your function**.

PRACTICAL FOR WEEK FROM DAY 1 TO 5

Spring Boot Project Setup, Maven Lifecycle & IDE Configuration Guide

To create a Spring Boot project, you use start.spring.io to generate and download a ready-made Maven project, then open it in your IDE. In IntelliJ, you simply open the folder, let it load the Maven configuration, and run the main class — that's usually enough. In Eclipse STS, you must manually import it as an "Existing Maven Project" because Eclipse does not automatically detect Maven projects, which is why your mentor followed that longer process. In general, all IDEs must recognize the project as a Maven project, download dependencies, and configure the classpath before running. IntelliJ automates most of this, while Eclipse requires more manual importing and configuration steps.

The Maven commands serve different purposes: mvn clean deletes old compiled files, mvn package compiles the project and creates a deployable .jar file, mvn spring-boot:run compiles (if needed) and starts the application directly, and mvn clean spring-boot:run first deletes old build files and then recompiles and runs the application in one command. The key difference between mvn spring-boot:run and mvn clean spring-boot:run is that the second command forces a fresh rebuild before starting, while the first one simply compiles changed files and runs. The difference between mvn package and mvn spring-boot:run is that package creates a jar file but does not start the application, whereas spring-boot:run starts the application without needing a jar file.

Your mentor likely used clean package and then clean spring-boot:run to demonstrate the full Maven lifecycle, ensure a completely fresh build, or follow a professional habit used in real projects. However, for everyday development — especially in IntelliJ — you can simply run the main class without manually running these Maven commands unless you need to create a jar for deployment, verify a clean build, or fix a build-related issue. Finally, in any IDE, the installed JDK version, the Java version selected in Spring Initializr, and the SDK configured inside the IDE must match (for example, all set to Java 17). If they do not align, you may face compilation or runtime errors even if everything else is correct.

Industry-Standard Spring Boot Project Structure & Coding Discipline

---

1️⃣ Naming Convention

Rename components as:

* `MyFirstAppController`

* `AddService` (inside interface package)

* `AddServiceImpl` (inside service implementation package)

2️⃣ Proper Layered Discipline (Industry Standard)

✅ Controller → Service Communication Rule

* The **Controller must talk to the Service layer using an interface reference only.**

* The controller should **never use the implementation class directly.**

* The controller must **not use a child reference.**

* Always program to an **interface, not an implementation.**

This ensures:

* Loose coupling

* Better testability

* Easy future replacement of implementations

* Clean architecture

3️⃣ Package Structure

Inside the `service` package:

```

service

 ├── interfaces (or api)
```

```
| └── AddService.java
└── impl
└── AddServiceImpl.java
```

🔹 AddService.java (Interface)

This contains the method declaration:

* `add()` method

🔹 AddServiceImpl.java

* Implements `AddService`
* Contains the actual business logic

4️⃣ Controller Implementation

MyFirstAppController

* Use `@RestController`
* Use `@GetMapping`
* Inject `AddService` using interface reference

Example logic flow:

Merchant → REST API → Controller → Service Interface → Service Implementation

---

5️⃣ Exposing the Controller

To allow merchants or external systems to call this controller:

* Use `@RestController`
* Use `@GetMapping` (or appropriate mapping annotation)

This exposes it as a REST API endpoint.

6️⃣ Industry-Level Additions (Short Summary – Mentor Recommended)

* **SonarQube** – Ensures code quality, detects bugs and code smells, and helps maintain clean, maintainable code in CI/CD pipelines.

* **GitHub Copilot** – Improves developer productivity by assisting with boilerplate and repetitive code while maintaining structure consistency.

* **Spring Boot DevTools** – Speeds up development with automatic restart and faster feedback during coding.

* **Logging (SLF4J + Spring AOP)** – Industry-standard logging practice. Avoid `System.out.println()`. Use SLF4J for proper log levels and configurability, and AOP to handle logging as a cross-cutting concern for cleaner business logic.

7️⃣ Summary (Mentor-Advised Industry Practice)

This structure and discipline were advised as real-world industry practices:

* Controller talks to Service via interface only

* Implementation hidden

* Clean layered architecture

* REST exposure for external systems

* Code quality enforcement via SonarQube

* Productivity tools like GitHub Copilot

* DevTools for faster development

* Proper logging using SLF4J + AOP

* Avoid `System.out`

This approach builds scalable, maintainable, and professional-grade Spring Boot applications.